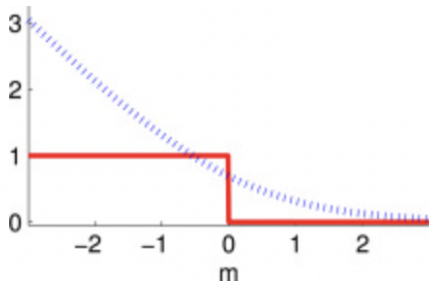


Perceptron

Loss Function Minimisation

Procheta Sen



Loss Function: Step Function

Let $\mathcal{D} = \{(\bar{X}_1, y_1), \dots, (\bar{X}_n, y_n)\}$ be the training dataset, where
 $\bar{X}_k = (x_k^{(1)}, x_k^{(2)}, \dots, x_k^{(d)})^T$ for every $k = 1, \dots, n$.

Let

$$a_k = b + \sum_{i=1}^d w_i x_k^{(i)}.$$

Define **loss function** on a single training object $(\bar{X}_k, y_k)$ as

$L(b, \bar{W}, \bar{X}_k, y_k) = 1$ if $\bar{X}_k$ is misclassified, and $L(b, \bar{W}, \bar{X}_k, y_k) = 0$, otherwise.

Define **loss function** for the training dataset $\mathcal{D}$ as

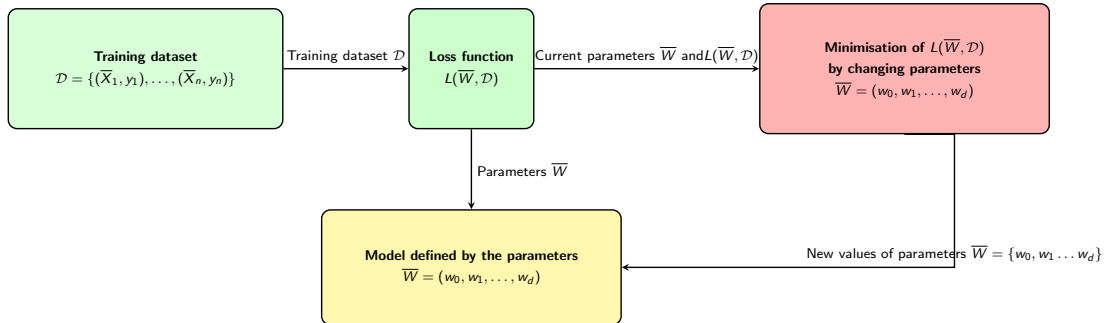
$$L(b, \bar{W}, \mathcal{D}) = \sum_{k=1}^n L(b, \bar{W}, \bar{X}_k, y_k) = \text{no. of misclassifications}.$$

- this function is piecewise-constant with many discontinuities;
- the derivative of this function (when it exists) is equal to 0;
Hence the gradient descent is not applicable!

Loss Function: Number of Misclassifications

Loss function for the training dataset $\mathcal{D}$ as

$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^n L(b, \overline{W}, \overline{X}_k, y_k) = \text{no. of misclassifications.}$$



Loss Function $h(t) = \max(0, t)$

Let $\mathcal{D} = \{(\bar{X}_1, y_1), \dots, (\bar{X}_n, y_n)\}$ be the training dataset, where

$$\bar{X}_k = (x_k^{(1)}, x_k^{(2)}, \dots, x_k^{(d)})^T \quad \text{for every } k = 1, \dots, n.$$

Let

$$a_k = b + \sum_{i=1}^d w_i x_k^{(i)}. \quad h(t) = \max(0, t).$$

Define **loss function** for a single object $\bar{X}_k$ as

$$L(b, \bar{W}, \bar{X}_k, y_k) = h(-y_k \cdot a_k).$$

Define **loss function** for the training dataset $\mathcal{D}$ as

$$L(b, \bar{W}, \mathcal{D}) = \sum_{k=1}^n L(b, \bar{W}, \bar{X}_k, y_k) = \sum_{k=1}^n h(-y_k \cdot a_k).$$

Note that

$$\begin{aligned} L(b, \bar{W}, \bar{X}_k, y_k) &= 0, & \text{if } \bar{X}_k \text{ is classified correctly;} \\ L(b, \bar{W}, \bar{X}_k, y_k) &= -y_k \cdot a_k \geq 0, & \text{if } \bar{X}_k \text{ is misclassified.} \end{aligned}$$

Hence the more misclassifications the model (with parameters b and $\bar{W}$) does, the larger the **loss function** for the training dataset $L(b, \bar{W}, \mathcal{D})$ becomes.

We want to find the parameters b and $\bar{W}$ that minimise $L(b, \bar{W}, \mathcal{D})$!

Loss Function Minimisation

$$L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^n L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^n h(-y_k \cdot a_k)$$

Use the gradient descent method:

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T - \mu \nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \mathcal{D})$$

$$\nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^n \nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^n \nabla_{b, w_1, \dots, w_d} h(-y_k \cdot a_k)$$

Computation of $\nabla_{b,w_1,\dots,w_d} h(-y_k \cdot a_k)$

$$h'(t) = 0, \text{ when } t < 0$$

$h'(t) = 1$, when $t \geq 0$ (h is not differentiable at $t = 0$, but we can extend h' by setting $h'(0) = 1$)

$$\frac{\partial h(-y_k \cdot a_k)}{\partial b} = h'(-y_k \cdot a_k) \cdot \frac{\partial}{\partial b}(-y_k \cdot a_k) = -y_k$$

$$\frac{\partial h(-y_k \cdot a_k)}{\partial w_i} = h'(-y_k \cdot a_k) \cdot \frac{\partial}{\partial w_i}(-y_k \cdot a_k) = -y_k \cdot x_k^{(i)}$$

If $\bar{X}_k$ misclassified, then

$$\begin{aligned}\nabla_{b,w_1,\dots,w_d} h(-y_k \cdot a_k) &= \left(\frac{\partial h(-y_k \cdot a_k)}{\partial b}, \frac{\partial h(-y_k \cdot a_k)}{\partial w_1}, \dots, \frac{\partial h(-y_k \cdot a_k)}{\partial w_d} \right)^T \\ &= -y_k \cdot (1, x_k^{(1)}, x_k^{(2)}, \dots, x_k^{(d)})^T\end{aligned}$$

Otherwise

$$\nabla_{b,w_1,\dots,w_d} h(-y_k \cdot a_k) = (0, 0, 0, \dots, 0)^T$$

Loss Function Minimisation

$$\nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \mathcal{D}) = \sum_{k=1}^n \nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \overline{X}_k, y_k) = \sum_{k=1}^n \nabla_{b, w_1, \dots, w_d} h(-y_k \cdot a_k)$$

Use the gradient descent method:

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T + \mu \sum_{k: \overline{X}_k \text{ misc.}} y_k \cdot (1, x_k^{(1)}, x_k^{(2)}, \dots, x_k^{(d)})^T$$

Batch Gradient Descent

- to make a single update of the parameters we need to use **whole training dataset**
- extremely slow if we have a huge dataset

Online Gradient Descent

Idea: make parameter updates after each misclassification

Instead of

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T - \mu \nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \mathcal{D})$$

For a misclassified object $(\overline{X}_k, y_k)$ the update becomes

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T - \mu \nabla_{b, w_1, \dots, w_d} L(b, \overline{W}, \overline{X}_k, y_k)$$

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T - \mu \nabla_{b, w_1, \dots, w_d} h(-y_k \cdot a_k)$$

Update Rule for Perceptron

For a misclassified training object $(\bar{X}, y)$ with the activation score

$$a = b + \sum_{i=1}^d w_i x_i$$

the weights are updated as follows

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T - \mu \nabla_{b, w_1, \dots, w_d} h(-y \cdot a)$$

$$\nabla_{b, w_1, \dots, w_d} h(-y \cdot a) = -y \cdot (1, x_1, x_2, \dots, x_d)^T$$

$$(b, w_1, \dots, w_d)^T \leftarrow (b, w_1, \dots, w_d)^T + \mu y \cdot (1, x_1, x_2, \dots, x_d)^T$$

$$b \leftarrow b + \mu \cdot y$$

$$w_i \leftarrow w_i + \mu \cdot y \cdot x_i \quad \text{for all } i = 1, \dots, d$$

By setting $\mu = 1$ we obtain exactly the update rule for Perceptron.

The Training Algorithm

PerceptronTrain(Training data: D , MaxIter)

- 1: $w_i = 0$ for all $i = 1, \dots, d$;
- 2: $b = 0$
- 3: **for** iter = 1 ... MaxIter **do**
- 4: **for all** $(\bar{X}, y) \in D$ **do**
- 5: $a = \bar{W}^T \bar{X} + b$
- 6: **if** $y \cdot a \leq 0$ **then**
- 7: $w_i = w_i + y \cdot x_i$, for all $i = 1, \dots, d$
- 8: $b = b + y$
- 9: **return** $b, w_1, w_2, \dots, w_d$

For $\mu = 1$

$$w_i \leftarrow w_i + y \cdot x_i$$

$$b \leftarrow b + y$$